

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning Repository Link:

https://github.com/Atharsh2910/Deep_Learning_Lab/tree/main/Lab4

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27
Name	ATHARSH S U	
Register Number	24011101008	
Roll Number	24110049	
Class	AI-DS 'A'	

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogleNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogleNet (Inception Network)

GoogleNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogleNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

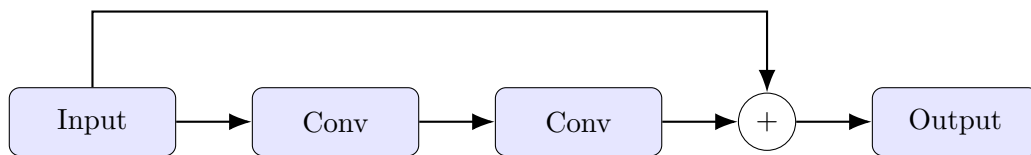


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis
- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

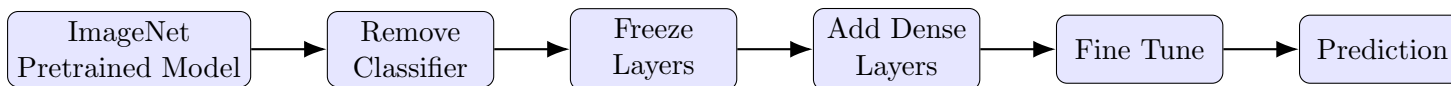


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

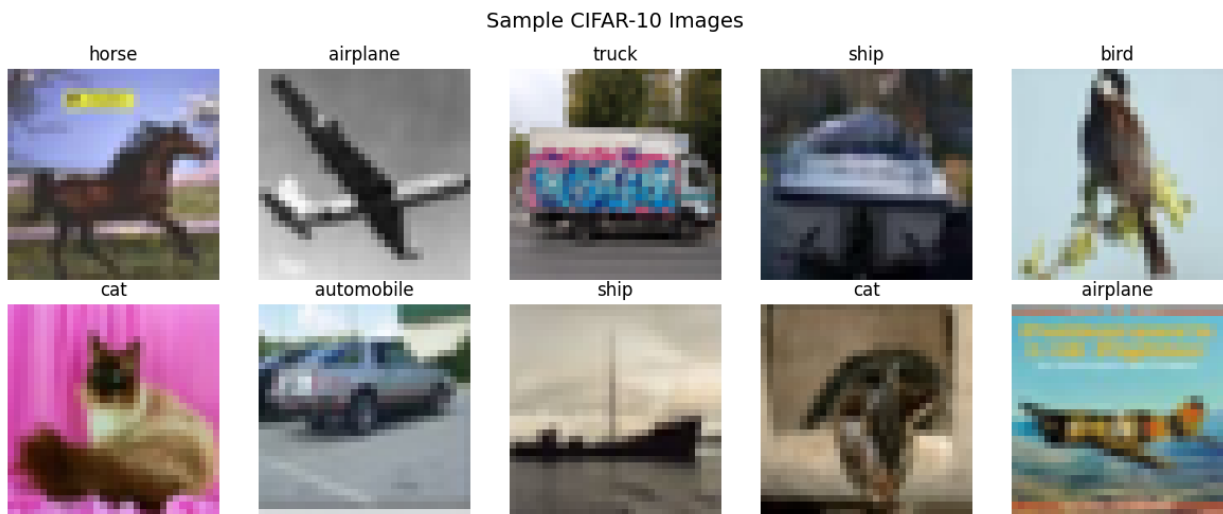
For this experiment the 50,000 training images were further split into a training set and a validation set (90:10) using a fixed random seed (`SEED = 42`) for reproducibility. The actual split obtained was

- Training set : 45,000 images, shape (45000, 32, 32, 3)
- Validation set : 5,000 images, shape (5000, 32, 32, 3)
- Testing set : 10,000 images, shape (10000, 32, 32, 3)
- Pixel value range after normalization : [0.000, 1.000]

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range [0,1].
3. Display ten sample images.
4. Print the dimensions of the training and testing datasets.



Inference: The sample images confirm that the CIFAR-10 images are low resolution (32×32) colour photographs with considerable variation in pose, background and lighting, which makes the classification task non-trivial even though only 10 classes are involved.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.

4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

In this implementation **VGG16** and **ResNet50** were both loaded with ImageNet weights (`include_top=False`) their convolutional bases frozen, and the same classification head attached: Global Average Pooling $\rightarrow$ Dense(ReLU) $\rightarrow$ Dropout(0.3) $\rightarrow$ Dense(10, Softmax). CIFAR-10 images were resized from 32×32 to 96×96 before being passed through the backbone-specific `preprocess_input` function. The VGG16 transfer-learning model summary reported

- Total parameters : 14,848,586 (56.64 MB)
- Trainable parameters (frozen stage) : 133,898 (523.04 KB)
- Non-trainable parameters (frozen stage) : 14,714,688 (56.13 MB)

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10–20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Implementation Details (Actual Configuration Used)

The experiment was executed on TensorFlow 2.20.0 with two GPUs (Tesla T4) available and mixed-precision training enabled (`mixed_float16`). The random seed was fixed to 42 for NumPy, Python and TensorFlow. The exact configuration dictionary used is summarised below.

Parameter	Value
Number of Classes	10
Validation Split	0.1
Transfer-Learning Image Size	96×96
Baseline (from-scratch) Image Size	32×32
Batch Size	64
Epochs – Baseline Models	12
Epochs – Frozen (Task 3)	8
Epochs – Fine-Tuning (Task 4)	6
Learning Rate – Frozen Stage	0.001
Learning Rate – Fine-Tuning Stage	1×10^{-5}
Epochs – Optimizer Comparison	5
Epochs – Hyperparameter Search	4
Loss Function	Sparse Categorical Cross-Entropy

Three CNNs were also trained **from scratch** (without transfer learning) on the native 32×32 CIFAR-10 images as baselines for comparison: a classic **LeNet-5**, a scaled-down **AlexNet (AlexNet-Mini)** and a scaled-down **ResNet (ResNetMini)** with a single residual block. All three baselines and the transfer-learning frozen stage were trained with the Adam optimizer.

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

For VGG16, the last 6 layers of the convolutional base (approximately the last convolution block) were unfrozen and trained for 6 further epochs with a reduced learning rate of 1×10^{-5} . For ResNet50, the last 10 layers of the convolutional base were unfrozen and trained for 6 further epochs with the same reduced learning rate.

Model	Accuracy Before Fine-Tuning	Accuracy After Fine-Tuning
VGG16	83.10%	90.26%
ResNet50	84.41%	86.14%

Inference: Fine-tuning the last convolution block improved test accuracy for both backbones. The improvement was much larger for VGG16 (+7.16 percentage points) than for ResNet50 (+1.73 percentage points), suggesting that VGG16’s ImageNet features benefited more from adapting to the low-resolution CIFAR-10 domain, while ResNet50’s residual features were already closer to optimal for this task even in the frozen state.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

All models were evaluated on the held-out 10,000-image CIFAR-10 test set using `accuracy_score`, `precision_score`, `recall_score`, `f1_score` (weighted average), `confusion_matrix` and `classification_report` from scikit-learn. The detailed results are reported in Section 18.

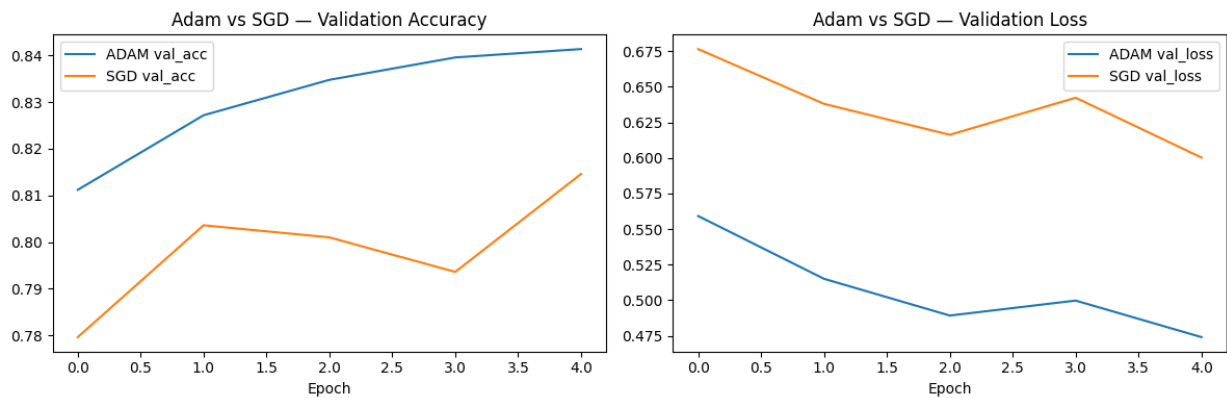
16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

16.1 Optimizer Comparison (VGG16, 5 epochs, 96×96 input)

Optimizer	Test Accuracy (%)	F1-score	Training Time (s)
Adam (lr = 0.001)	82.79	0.8278	89.0
SGD (lr = 0.01, momentum = 0.9)	80.79	0.8079	88.1



Inference: Adam converged faster and reached a lower validation loss and higher validation accuracy than SGD within the same epochs.

16.2 Hyperparameter Search Results (VGG16, 4 epochs each)

Learning Rate	Dense Units	Frozen Layers	Validation Accuracy	Time (s)
0.0010	128	All	0.8330	73.4
0.0010	256	All	0.8402	72.8
0.0001	256	Partial (last 4 layers unfrozen)	0.9098	89.0

Inference: Increasing the dense layer width from 128 to 256 units gave a small improvement in validation accuracy (+0.72 points) at almost no extra time cost. Partially unfreezing the convolutional base combined with a lower learning rate gave the largest gain (+6.96 points over the best fully-frozen configuration), confirming that allowing some backbone layers to adapt to CIFAR-10 is more valuable than simply widening the classifier head.

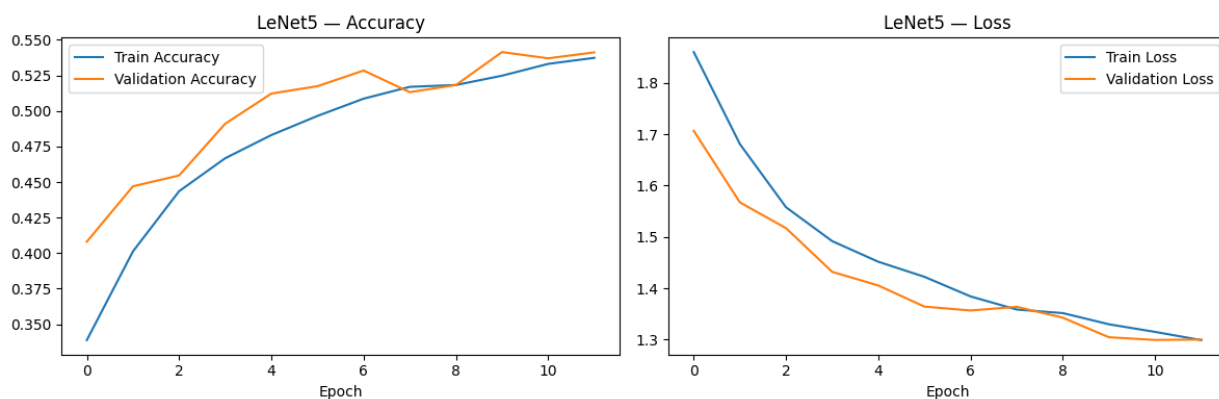
17. Mandatory Plots

Include the following plots in the laboratory report.

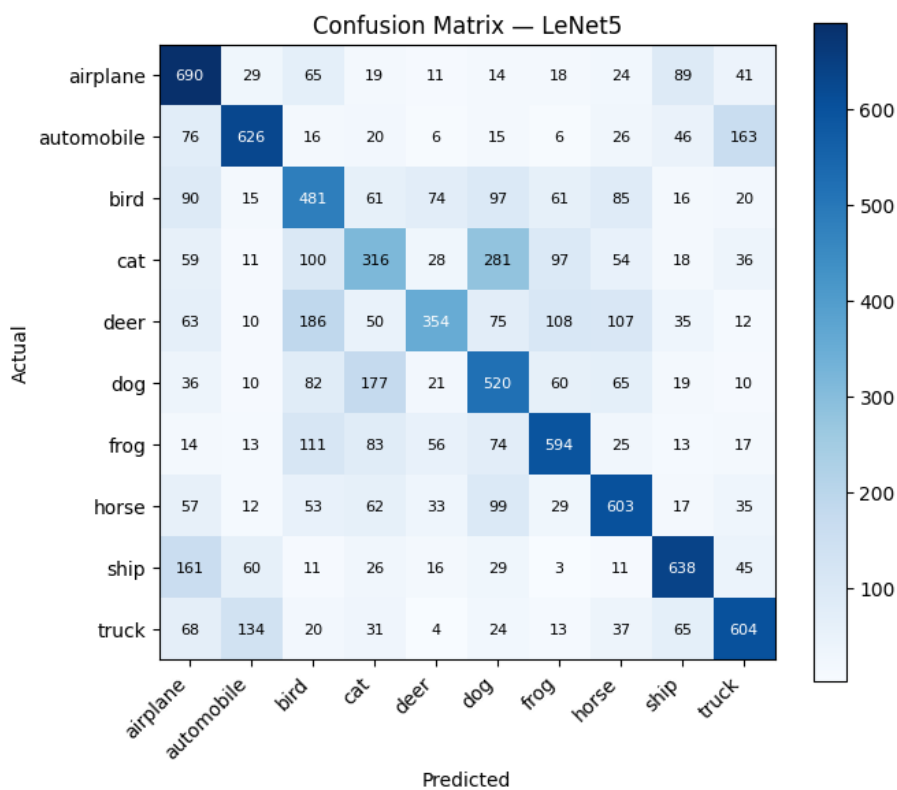
1. Sample CIFAR-10 images
2. Training Accuracy
3. Validation Accuracy
4. Training Loss
5. Validation Loss
6. Confusion Matrix
7. Misclassified Images (Optional)

Students should write a brief inference (2–3 lines) for each figure.

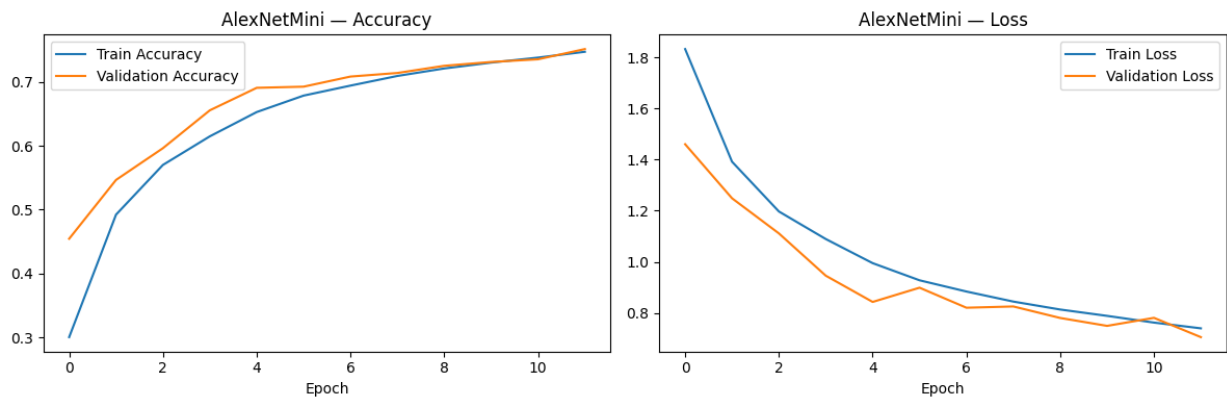
17.1 Baseline (From-Scratch) Models



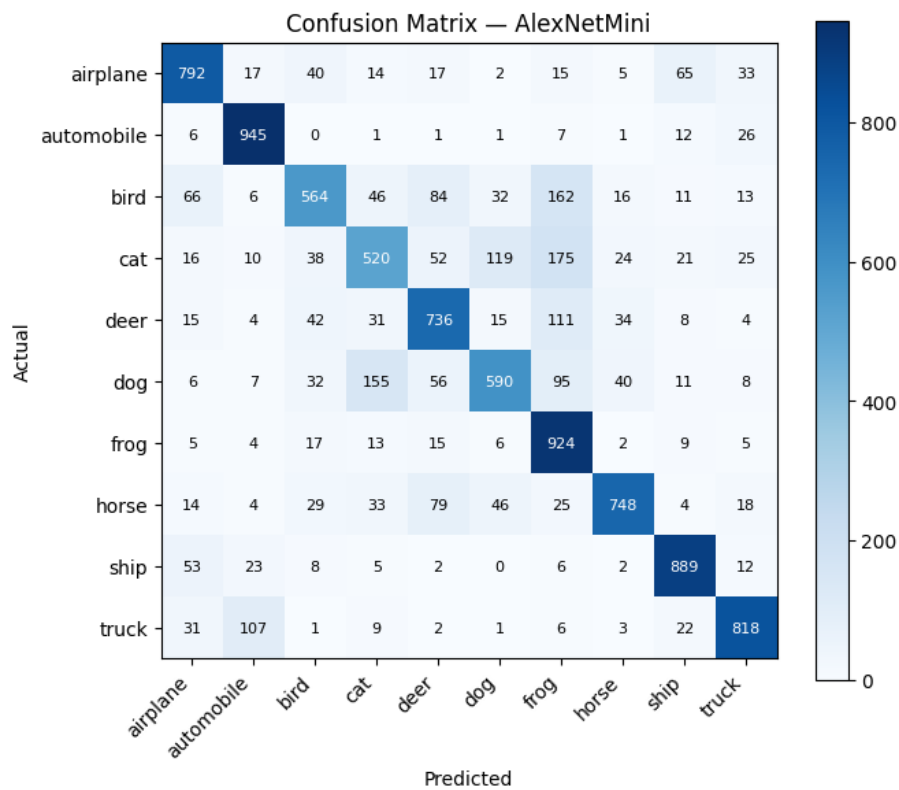
Inference: LeNet-5's training and validation accuracy increase slowly and converges across iterations, with no overfitting, but the final accuracy plateaus around 54%, reflecting the model's lower capacity.



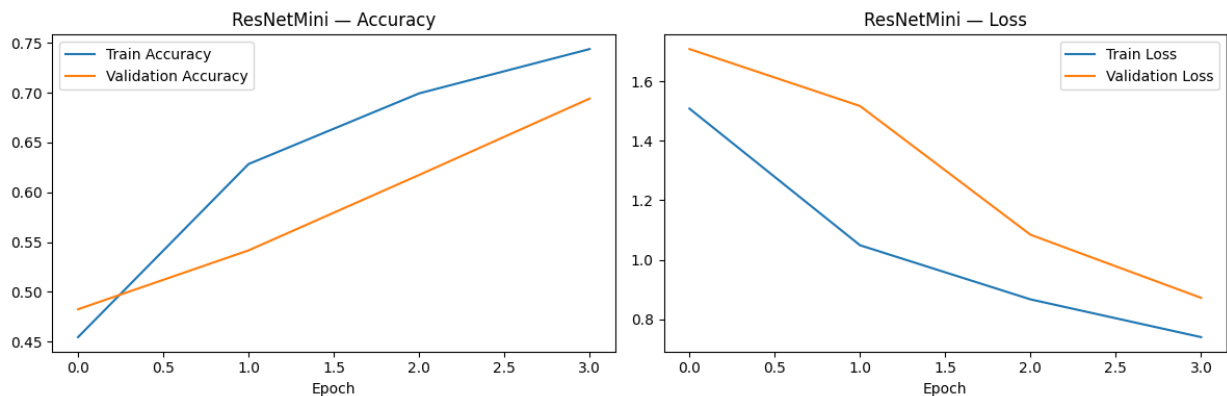
Inference: LeNet-5 could not classify visually similar classes such as cat/dog and automobile/truck, but performs relatively well on more distinctive classes such as ship and automobile.



Inference: AlexNetMini shows a steady increase in accuracy and decrease in loss, reaching around 75% training and validation accuracy by epoch 12, indicating good generalisation.

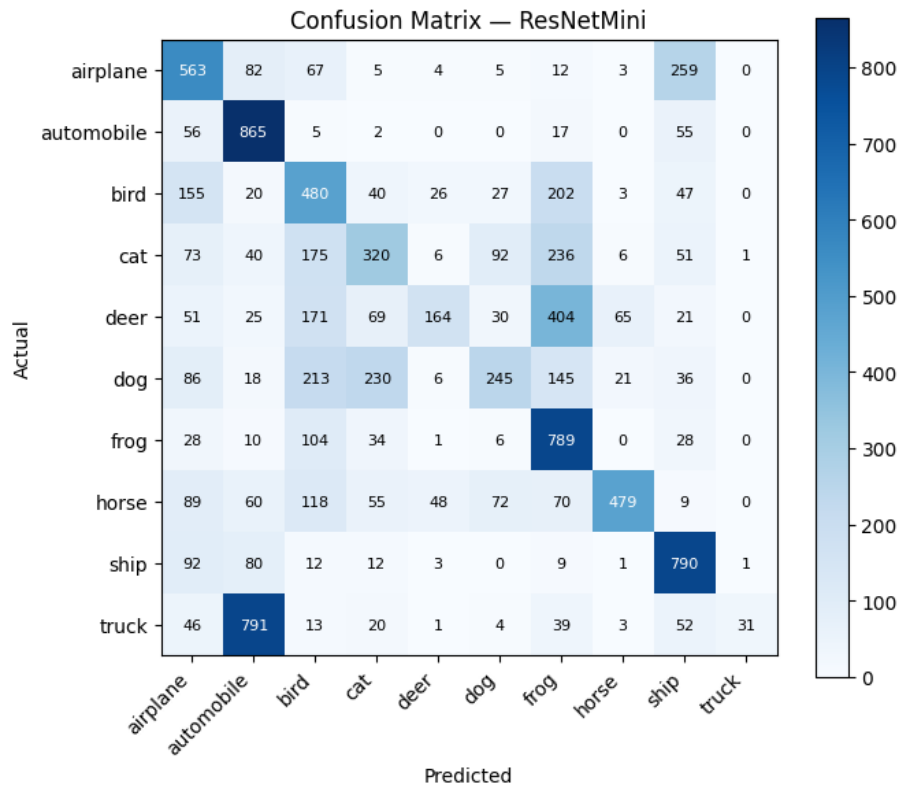


Inference: AlexNetMini achieves good classification across most classes, with the main confusion remaining between cat, dog and bird, which have lesser distinctive feature.



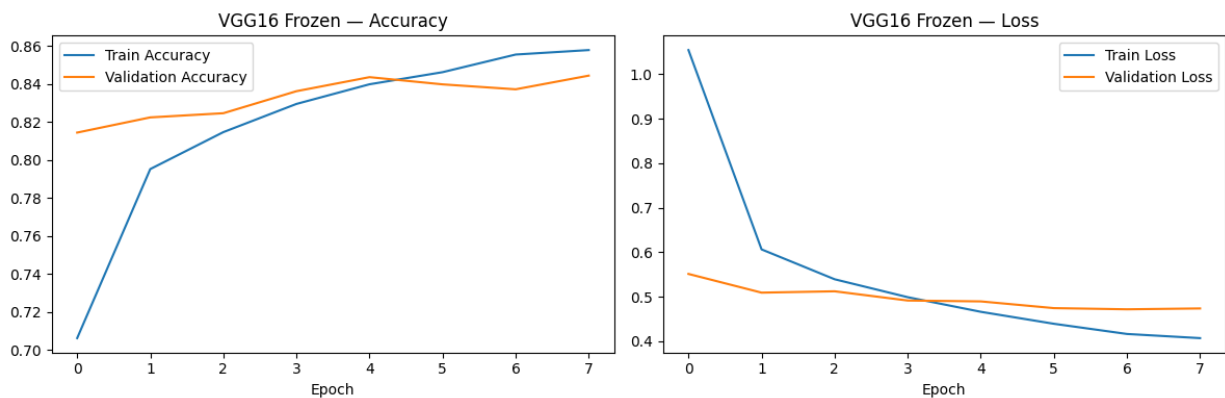
Inference: Training for ResNetMini was stopped early once validation loss stopped improving; the

restored weights generalised poorly to the test set (47.26% accuracy), suggesting the small residual model was not strong enough to be trained stably in so few epochs.

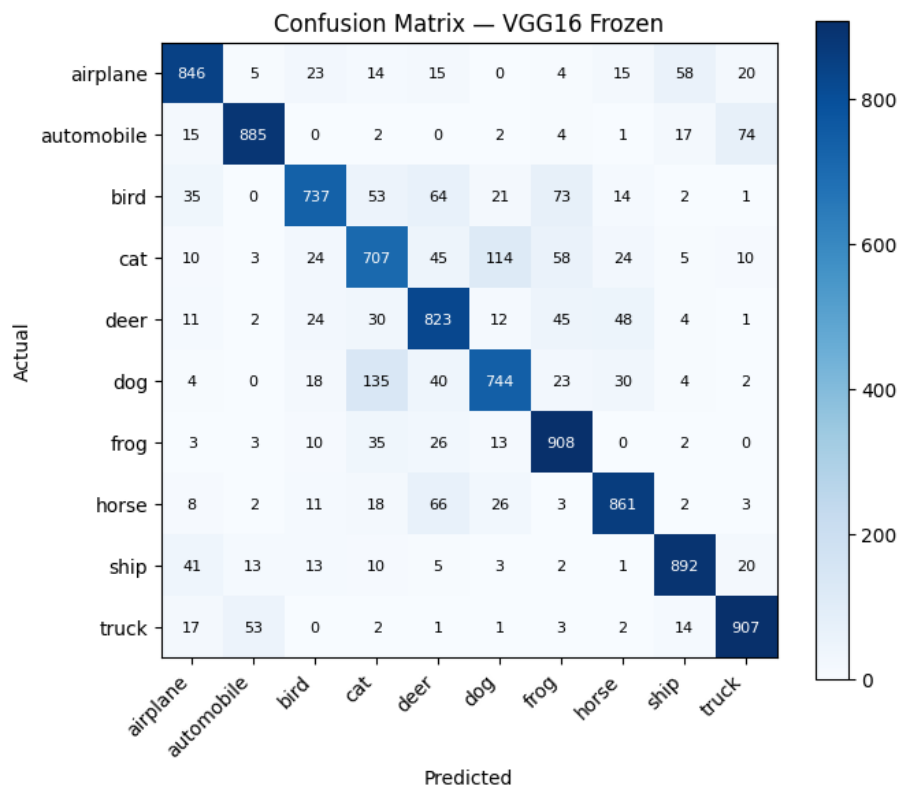


Inference: ResNetMini’s confusion matrix shows heavy bias toward a few classes, with the truck class predicted poorly, proving the unstable, under-trained behaviour.

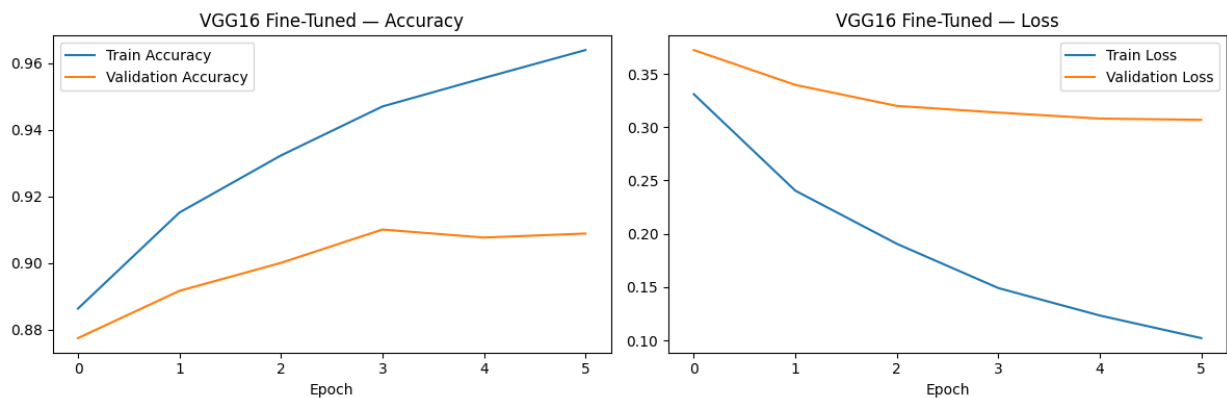
17.2 Transfer Learning – VGG16



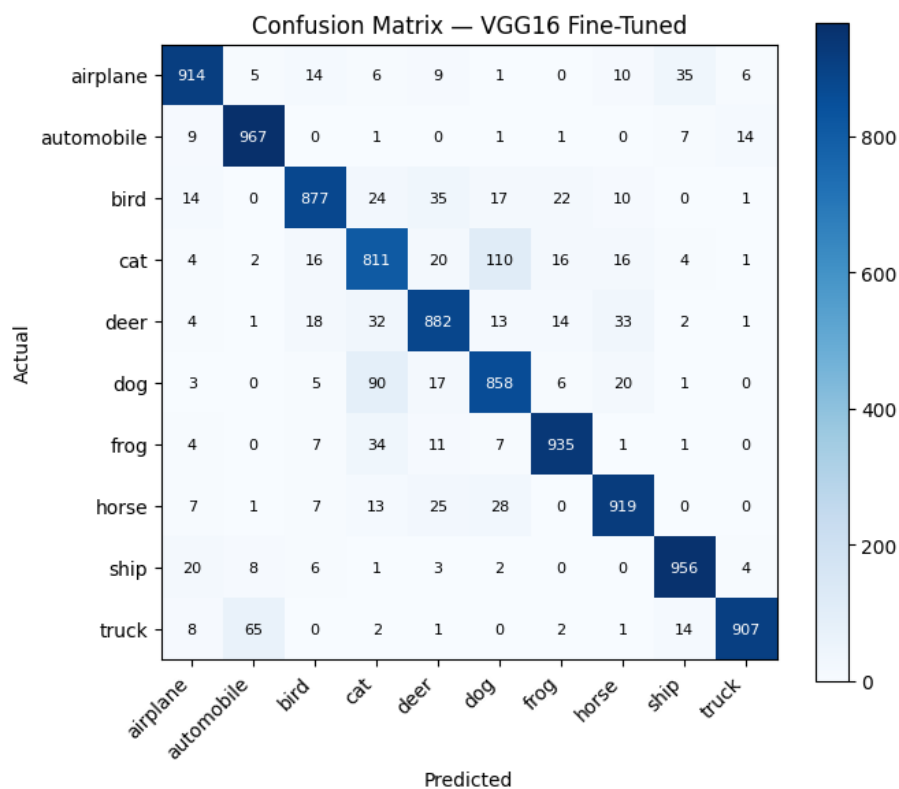
Inference: With the convolutional base frozen, VGG16 already reaches roughly 84–86% training accuracy and 83–84% validation accuracy within 8 epochs.



Inference: The frozen VGG16 model is strong across nearly all classes, with the largest remaining confusion between cat and dog.



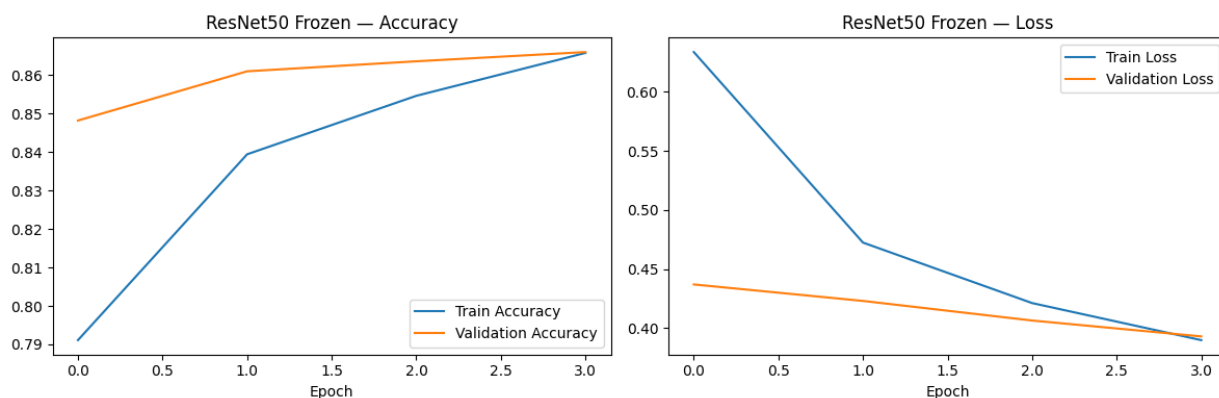
Inference: After unfreezing the last convolution block, training accuracy increases to 96% while validation accuracy reaches about 91%, indicating mild overfitting.



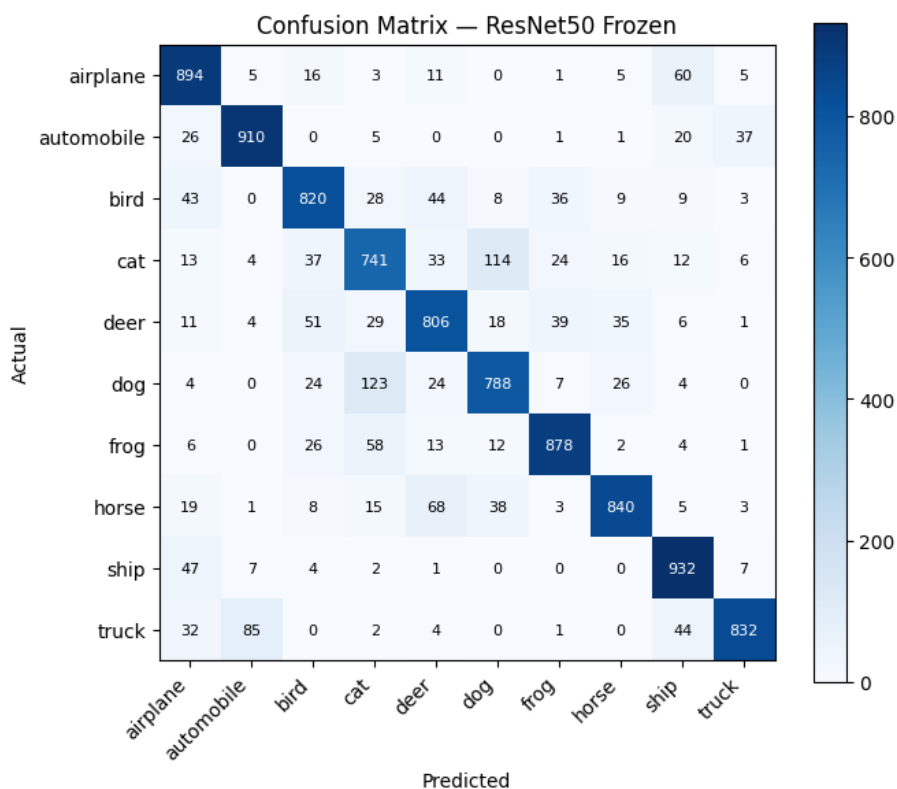
Inference: Fine-tuning sharpens the diagonal of the confusion matrix considerably compared to the frozen model, especially for automobile, ship and truck, which now exceed 90% recall.



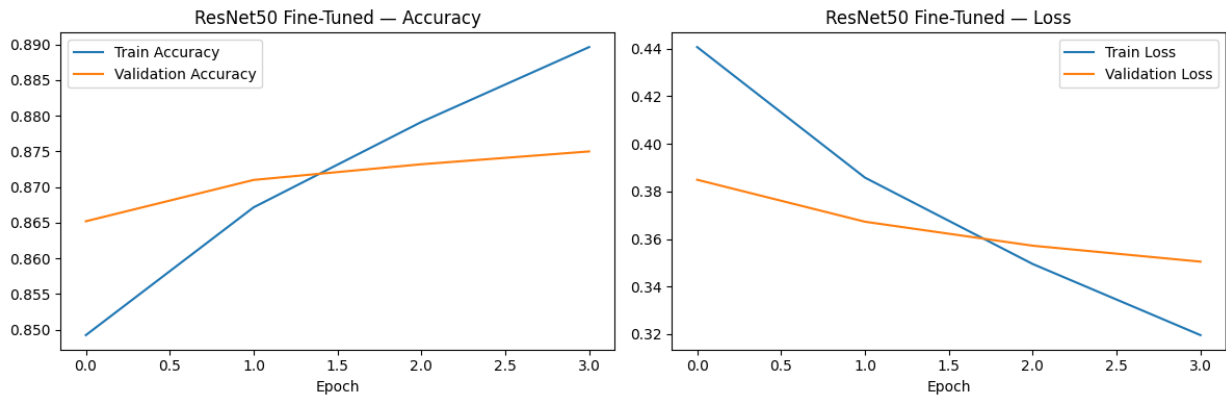
17.3 Transfer Learning – ResNet50



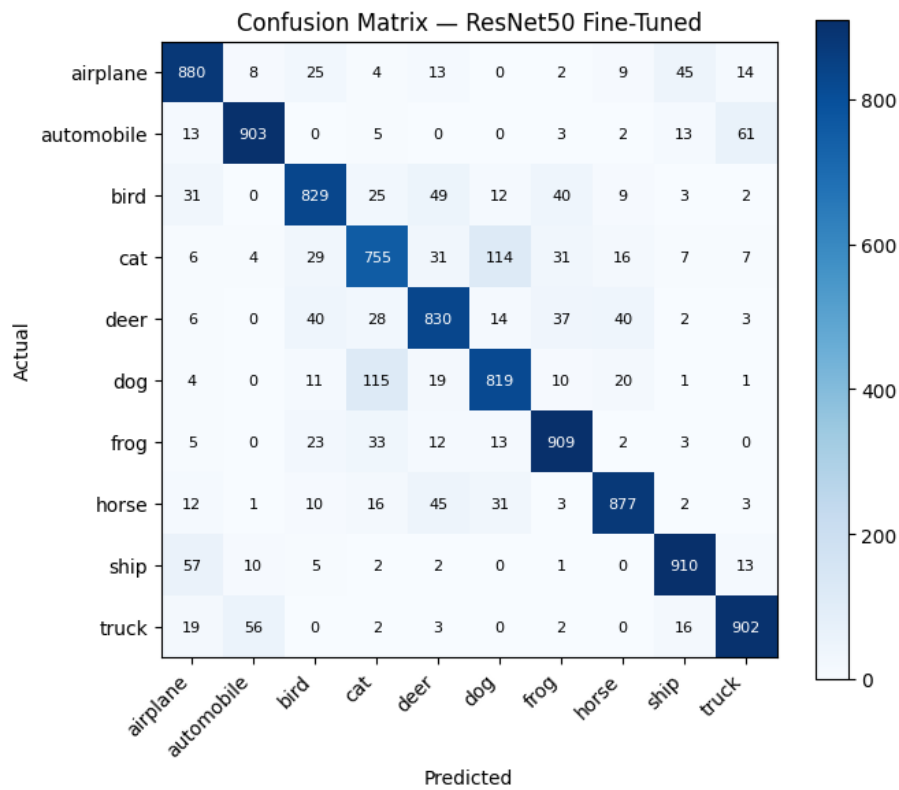
Inference: ResNet50's frozen backbone converges faster than VGG16's, reaching about 86-87% validation accuracy within just 4 epochs, proving that ResNet 50 is better than VGG16.



Inference: The frozen ResNet50 model gives a confusion matrix with high recall across most classes; cat and dog remain the most confused pair.

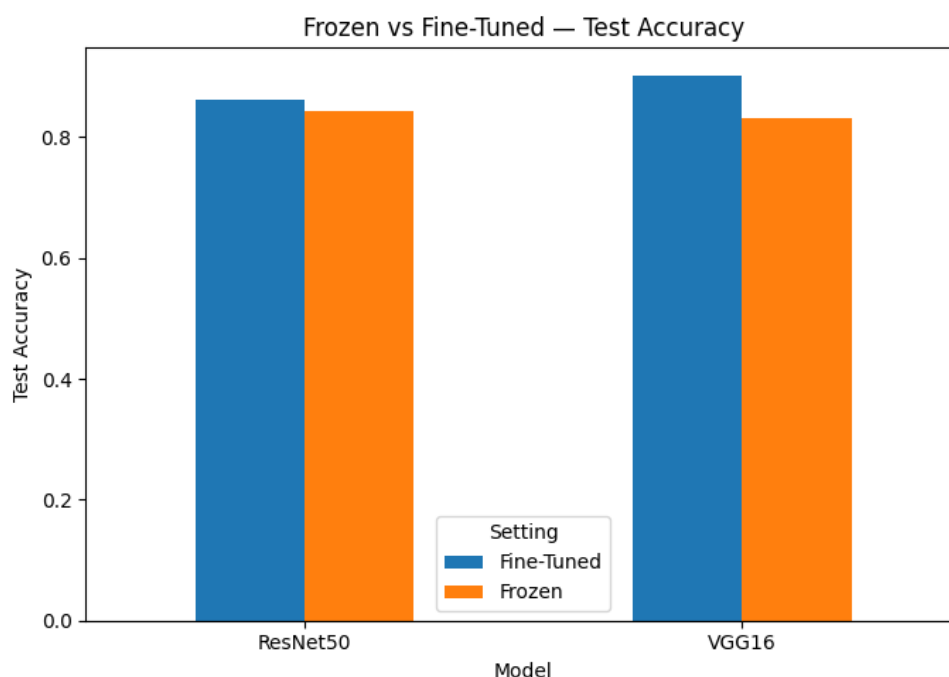


Inference: Fine-tuning the last 10 layers of ResNet50 gives a minimal additional gain than for VGG16, as the training and validation curves rise only little beyond the frozen-stage result, suggesting ResNet50’s frozen features were already close to their maximum performance.



Inference: Fine-tuning improves ResNet50’s precision and recall slightly across almost every class compared to the frozen model, with the largest gains for the cat and dog classes.

17.4 Overall Comparison



Inference: Fine-tuning improves test accuracy for both backbones, VGG16 gains substantially from fine-tuning and overtakes ResNet50, whereas ResNet50's frozen accuracy was already higher than VGG16's and it gains comparatively little from fine-tuning.

18. Results

18.1 Performance Metrics (Best Model: VGG16, Fine-Tuned)

Metric	Value
Training Accuracy	96.39%
Testing Accuracy	90.26%
Precision	0.9035
Recall	0.9026
F1-score	0.9028
Training Time	290.3 s (fine-tuning stage; 141.1 s frozen stage)
Total Parameters	14,848,586

18.2 Comparison of CNN Architectures (Experimental Results)

Model	Parameters	Test Accuracy (%)	Precision	Recall	F1-score	Training Time
LeNet-5	83,126	54.26	0.5481	0.5426	0.5403	232.8
AlexNetMini	6,976,842	75.26	0.7575	0.7526	0.7486	306.1
ResNetMini	681,226	47.26	0.5552	0.4726	0.4317	141.1
VGG16 (Frozen, TL)	14,848,586	83.10	0.8323	0.8310	0.8306	141.1
VGG16 (Fine-Tuned, TL)	14,848,586	90.26	0.9035	0.9026	0.9028	290.3
ResNet50 (Frozen, TL)	–	84.41	0.8455	0.8441	0.8440	91.1
ResNet50 (Fine-Tuned, TL)	–	86.14	0.8614	0.8614	0.8613	194.3

19. Discussion Questions

Answer the following questions.

- Why is AlexNet considered a breakthrough in deep learning?
AlexNet was considered a major breakthrough in Deep Learning, as it introduced the usage of ReLU activation function, introduction of dropouts, data augmentation, multiple CNN layers and parallel GPU training. These features of AlexNet improved the performance led to faster convergence, with a higher accuracy.
- Why does VGG16 use only 3×3 convolution filters?
VGG Net uses 3×3 filter because, it is proved efficient than larger filters by reducing the parameter size and using multiple 3×3 filter also increases the receptive field. VGG16 achieved good feature extraction using the 3×3 filter, proving its efficiency over larger filters.
- Explain the advantages of the Inception module.
Inception module simultaneously uses multiple filters of different sizes, trains the model and is stacked together. The usage of 1×1 filter produces feature map that contains the most important feature. Thus Inception Module, provides a better feature extraction, pooling and better feature maps, keeping the computation less intensive.
- What is the purpose of residual learning?
Residual Learning aids in skipping the connections that are not relevant for the training. It allows gradients to flow directly through shortcut connections, thereby reducing the vanishing gradient problem.
- Differentiate LeNet and ResNet.
LeNet is a primitive early stage CNN Network that has a shallow network with fewer layers. It works using convolution technique and pooling. On the other hand, ResNet is a much deeper architecture designed to train efficiently using residual connections. It has a deeper network with multiple layers, helping for large scale image recognition tasks. It uses skip connection which a key aspect of ResNet.
- What is Transfer Learning?
Transfer Learning is a technique where a model trained on one task is reused for another related task. Instead of training a model from scratch, the pretrained model's learned features are reused. It reduces the training time at the same time provides a better accuracy.
- Why is fine tuning required?
Fine-tuning is required because a pretrained model was originally trained for a different task. Fine-tuning helps in improving the accuracy for a task on top of the baseline accuracy while retaining the knowledge of pretrained model.
- Explain the difference between dilated convolution and transpose convolution.
A dilated convolution inserts spaces between kernel elements. It increases the receptive field without increasing the number of parameters. Transpose convolution, on the other hand, increases the spatial dimensions of feature maps.
- Why do pretrained models converge faster?
Pretrained models converges faster because they already have useful parameters learned from the previous dataset. Within a fewer epochs, a higher accuracy could be achieved due the prior

knowledge of the task.

10. Compare the computational complexity of LeNet and ResNet.

LeNet is computationally less intensive as it has fewer layers and lesser parameters. ResNet on the other hand, contains many layers, has multiple convolutional blocks, and uses for processing huge dataset. Thus ResNet is computationally more intensive.

20. Additional Exercises

1. Implement transfer learning using VGG16.
2. Repeat the experiment using ResNet50.
3. Compare Adam and SGD optimizers.
4. Compare training with frozen layers and fine tuning.
5. Evaluate the model using Precision, Recall and F1-score.
6. Compare the performance of LeNet, AlexNet and ResNet.

21. References

1. Yann LeCun, Leon Bottou, Yoshua Bengio and Patrick Haffner, *Gradient-Based Learning Applied to Document Recognition*, Proceedings of the IEEE, 1998.
2. Alex Krizhevsky, Ilya Sutskever and Geoffrey Hinton, *ImageNet Classification with Deep Convolutional Neural Networks*, NeurIPS, 2012.
3. Karen Simonyan and Andrew Zisserman, *Very Deep Convolutional Networks for Large-Scale Image Recognition*, ICLR, 2015.
4. Christian Szegedy et al., *Going Deeper with Convolutions*, CVPR, 2015.
5. Kaiming He, Xiangyu Zhang, Shaoqing Ren and Jian Sun, *Deep Residual Learning for Image Recognition*, CVPR, 2016.
6. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
7. TensorFlow Documentation: <https://www.tensorflow.org>
8. Keras Documentation: <https://keras.io>